

PhotonWeave

Lumerical FDTD를 대체하는 GPU 가속 FDTD 워크벤치

브라우저 CAD + Python API + NVIDIA CUDA 네이티브 솔버 · MIT 라이선스 · v0.14.0.dev

The screenshot displays the PhotonWeave web interface for a 2D TMz waveguide simulation. The interface is divided into several panels:

- Top Bar:** Includes the PhotonWeave logo, "FDTD SIN waveguide | 2D TMz", and a "CPU solver" indicator.
- Menu Bar:** Contains "File", "Edit", "View", "Materials", "Simulation", "Feature checklist", "Flux results", and "Help".
- Design Panel:** Features tabs for "Design", "FDTD", and "View". It includes icons for opening, saving, inspecting, and running simulations, as well as tools for creating structures like rectangles, circles, rings, spheres, and polygons.
- Objects Tree:** Lists the simulation components: "model" (containing "FDTD"), "STRUCTURES" (containing "waveguide"), "SOURCES" (containing "source"), "MONITORS" (containing "input" and "output"), and "Results View".
- Field Visualizer:** Shows three views of the simulation: "XY plane", "Perspective", and "YZ plane". The "XY plane" view shows a 2D cross-section of the waveguide with a red dot representing the source and yellow dots representing monitors. The "Perspective" view shows a 3D isometric view of the waveguide. The "YZ plane" view shows a 2D cross-section of the waveguide.
- Object Properties Panel:** Contains settings for the "FDTD" simulation region, including "General" (dimension: 2D (XY), resource: GPU if available, precision: float32, CUDA kernel: PyTorch reference, Frequency monitor kernel: PyTorch reference), "Geometry" (x span: 8 μm, y span: 6 μm, z span: 2 μm), and "Mesh settings" (mesh type: Uniform, independent axis spacing: dx = dy = dz = 0.05 μm, interface method: Staircase, interface sampling: Cell centers (legacy)).

왜 직접 만들었나
워크벤치: 무엇을 만들었나
엔진 내부
정확도 검증
성능
미분 가능 FDTD와 역설계
상호운용, 현황, 다음 단계

한 줄 요약

상용 FDTD 워크플로를 그대로
쓰면서, 계산은 GPU에서, 설계 루프는
Python에서 자동으로 도는 솔버를
직접 만들었습니다.

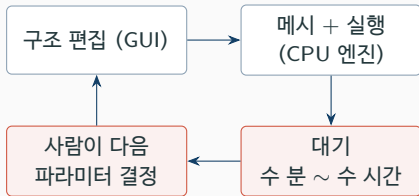
모든 속도 수치는 이 저장소에 기록된 측정값이며,
측정 조건과 한계를 슬라이드마다 같이
적었습니다.

왜 직접 만들었나

우리 연구에서 FDTD가 병목이 되는 지점

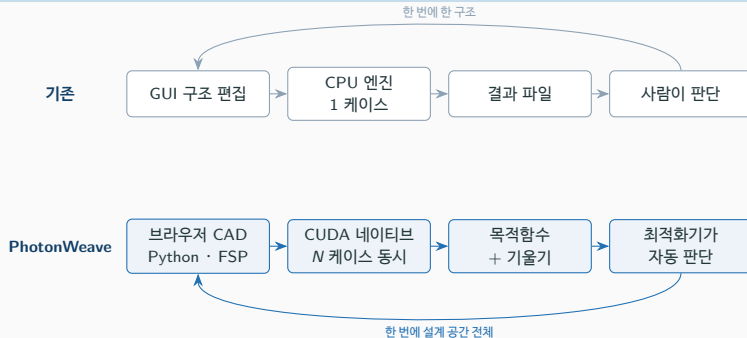
상용 FDTD의 실제 사용 패턴

- 구조 하나 계산 → 결과 확인 → 파라미터 수정 → 재계산
- 스윙/역설계는 **케이스 수만큼 순차 실행**
- 동시 실행 개수가 보유 라이선스에 묶임
- 우리가 비교 기록을 가진 구성은 **CPU 16 스레드 엔진** (상용 GPU 엔진은 미비교)
- 목적함수의 기울기 (gradient)를 사용자 코드에서 받을 수 없음



루프 한 바퀴가 느리면
설계 공간을 탐색할 수 없음

그래서 무엇을 바꿨나



익숙한 UI

Objects Tree,
CAD 4뷰, Layout/
Analysis 모드

GPU 네이티브

Yee/CPML fused
CUDA 커널 +
CUDA Graph

미분 가능

discrete adjoint로
 $\partial f / \partial \epsilon$ 직접 계산

핵심 수치 한 장 요약

6.5 – 15.2×

Lumerical CPU(16 threads) 대비 기록된 wall time 비

과거 빌드 · 동

일정확도 아님

9.6 – 17.1×

동일 격자·소스·모니터에서
오픈소스 GPU 기준선 대비

≈ 40×

같은 4케이스 배치:

NumPy CPU 47.04

s → CUDA 1.175 s

46.7 / 85.9×

adjoint 기울기 1회:

CPU 대비 GPU resident

(FP32 / complex FP64)

54 GiB

48 GB VRAM을 넘는

상태를 streaming으로

전진+역전파 완료

4.4e-6

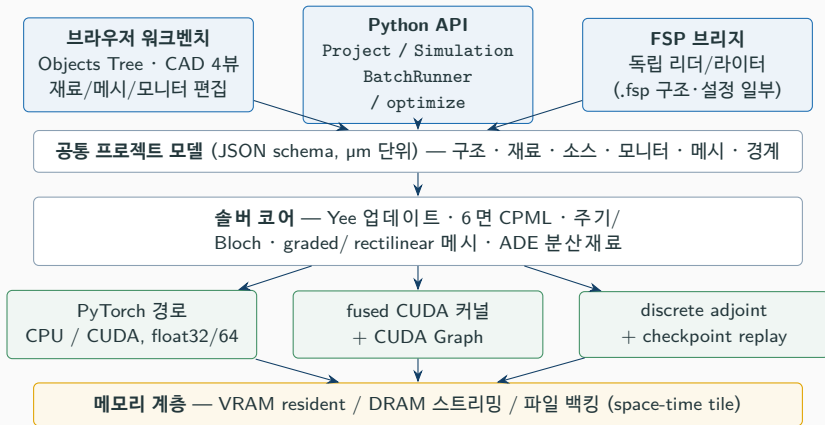
박막 해석해 대비 $R + T - 1$

잔차 (오늘 CPU에서 재현)

각 수치의 측정 조건·베이스라인·남은 검증 항목은 해당 슬라이드에서 그대로 밝힙니다.

워크벤처: 무엇을 만들었나

시스템 구조



그리드/백엔드 기반은 오픈소스 flaport/fdtd(MIT)를 사용하고, Yee 미분·CPML·주기/Bloch·CUDA Graph 실행은 직접 구현했습니다.

익숙한 화면 그대로: Layout 모드

PhotonWeave

FDTD

SIN waveguide | 2D TMz

CPU solver

File

Edit

View

Materials

Simulation

Feature checklist

Flux results

Help

Design

FDTD

View

Open

Save

FSP inspect

FSP

GPU

Python

Rectangle

Circle

Ring

Sphere

Polygon

FDTD region

Dipole

Sheet source

TFSF box

Time monitor

DFT / Flux

Duplicate

Delete

Fit view

Layout

Run

Stop

Run simulation

Objects Tree

Layout editor

Field visualizer

model

FDTD

STRUCTURES

waveguide

SOURCES

source

MONITORS

input

output

Results View

2D: simulation at $x = 0$

2D: simulation at $x = 0$

EXAMPLE PROJECTS

SIN waveguide

Cylinder scattering

Dielectric sphere

Simulation log

Python script

Export py

Object properties

FDTD

Simulation region

General

dimension

resource

precision

CUDA kernel

Frequency monitor kernel

Geometry

x span

y span

z span

Mesh settings

mesh type

Independent axis spacing

dx = dy = dz

interface method

interface sampling

SIMULATION SUMMARY

160 × 120 × 1

10,000 cells - 10.7 MB

At 0.1288 to 116.8 to 1000

source: simulation ends before the Gaussian pulse tail. Increase simulation time.

계산이 끝나면 그대로 Analysis 모드

PhotonWeave

FDTD

SIN waveguide | 2D TMz

CPU solver

vm

File

Edit

View

Materials

Simulation

Feature checklist

Flux results

Help

Design

FDTD

View

Open

Save

FSP inspect

FSP ... GPU

Python

Rectangle

Circle

Ring

Sphere

Polygon

FDTD region

Dipole

Sheet source

TFSF box

Time monitor

OFT / Flux

Duplicate

Delete

Fit view

Layout

Run

Stop

Geometry and wavelength in μm

Objects Tree

Layout editor

Field visualizer

model

FDTD

STRUCTURES

waveguide

SOURCES

source

MONITORS

input

output

Results View

Ez field snapshots

input - Ez

output - Ez

0.35 s solver loop

CPU

33.0 Mcells/s

NumPy CPU

Step limit reached

600 / 600 steps

Simulation log

Python script

EXAMPLE PROJECTS

SIN waveguide

Cylinder scattering

Dielectric sphere

Ez · real · XY plane

Step 600

NPZ

Ez · real · $\pm 5.37\text{e-1}$ (reduced field)

Point monitor: input - Ez

Time signal

Field spectrum

CSV

Simulation log

Python script

Object properties

Preview simulation mesh

You sampling places materials at each electric field component. Staircase interfaces follow the grid. Test mesh convergence for the required accuracy.

Boundary conditions

PML layers

10 cells

x min bc

PML

x max bc

PML

y min bc

PML

y max bc

PML

background index

1

Cyclic boundaries are paired. Bloch phase is the phase gained over one positive unit-cell translation. PML coefficients use the native CPML convention.

Simulation time

dt stability factor

0.99

time steps

600

SIMULATION SUMMARY

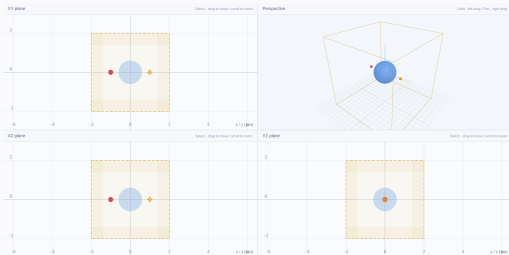
160 × 120 × 1

10,200 cells · 33.7 MBS

0.01088 fs · 70.1 fs total

source: simulation ends before the Gaussian pulse tail. Increase simulation time.

같은 프로젝트를 세 가지 입구에서



3D 예제(유전체 구)의 4뷰 CAD. 브라우저에서 만든 구조가 그대로 Python 객체입니다.

```
from photonweave import (Project, Region,  
                          Structure, Source, Monitor, Simulation)
```

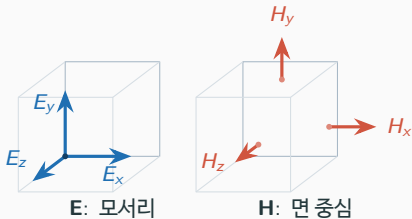
```
project = Project(  
    name="My waveguide",  
    region=Region(size=(8, 6, 2), mesh=0.05,  
                  steps=1000, backend="cuda"),  
    structures=[Structure(name="core",  
                          size=(8, 0.65, 0.4))],  
    sources=[Source(center=(-2.5, 0, 0),  
                    wavelength=1.55)],  
    monitors=[Monitor(name="output",  
                      center=(2, 0, 0))])
```

```
result = Simulation(project).run()  
result.save("results/run.npz")
```

UI → JSON → Python이 같은 모델. UI의 Export .py가 실행 가능한 스크립트를 그대로 만들어 줍니다.

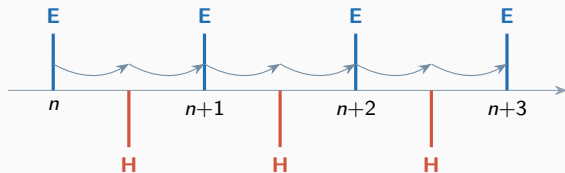
엔진 내부

Yee 격자와 leapfrog 시간 전개



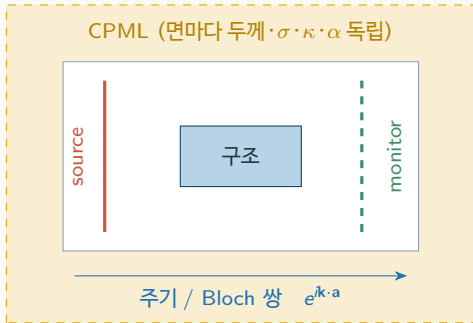
$$\epsilon \frac{\partial \mathbf{E}}{\partial t} = \nabla \times \mathbf{H}, \quad \mu \frac{\partial \mathbf{H}}{\partial t} = -\nabla \times \mathbf{E}$$

E는 모서리, H는 면 중심. 2차 정확도 중심차분, CFL
안정계수 기본 0.99.



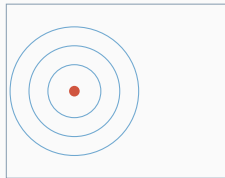
- 3D 6성분 + 2D(z-불변) TMz/TEz
- float32 / float64, 실수장과 복소장 (Bloch) 모두
- 자기장 소스는 H half-step 시각에 주입
- CUDA Graph로 스텝 커널 launch 오버헤드 제거

경계 조건: 6면 CPML과 주기/Bloch

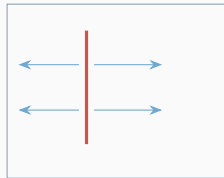


- **Convolutional PML:** 3D 6면 / 2D 4면, 면별 레이어 수와 다항식 차수 독립 설정
- **Periodic / Bloch 쌍:** 복소 E/H를 유지하고, sheet 소스가 지정 Bloch 위상을 자동 적용
- Bloch 스냅샷은 real / imag / magnitude / phase 선택, NPZ는 완전한 복소 데이터 보존
- **아직 없음:** 대칭/반대칭 경계, PEC/PMC

경계 규약과 검증은 docs/BOUNDARIES.md에 정리되어 있습니다.



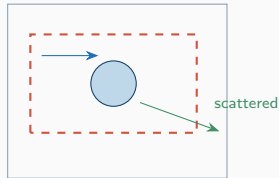
점 dipole (E/H)



soft sheet (양방향)



one-way 주기 평면

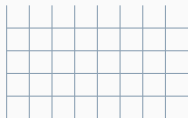


closed TFSF box

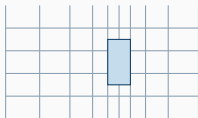
- Gaussian / 연속파 / 샘플 파형, Cartesian 또는 θ, ϕ 방향
- 자동 파장·주파수 범위 지정 (broadband), chirp, endpoint tapering
- one-way와 TFSF는 **현재 수직 입사**만 지원 — 경사 입사와 도파로 모드 소스는 미구현

TFSF는 산란 단면적을 Mie 해석해와 비교해 검증했습니다 (뒤 슬라이드).

메시: uniform · graded · 축별 독립 (rectilinear)



uniform



graded

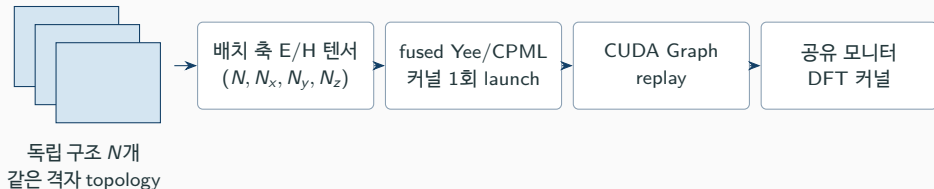


rectilinear ($dx \neq dy$)

- 축별 독립 간격과 명시적 node 배열, 직사각형 CFL, 물리 두께 기준 PML
- 횡방향으로 균일한 문제 (충 구조, 박막)에서는 **4.33–13.39×** 배치 시간 감소
- 같은 실제 시간 스텝, 같은 관측량에서 **72개 정확도 게이트 모두 통과** (최대 상대 L2 $5.95e-15$)
- 재료 경계는 기본 staircase, 실험적 subpixel 연산자 선택 가능

출처: README.md · Rectilinear mesh and batch ablation

CUDA 실행 모델: 왜 빨라지는가



커널 융합

E/H 업데이트 · CPML · 소스 ·
트레이스를 한 커널로. PyTorch
참조 경로 대비 **3.67–5.76×**

배치 축

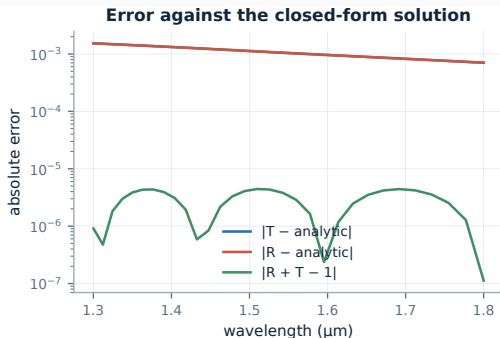
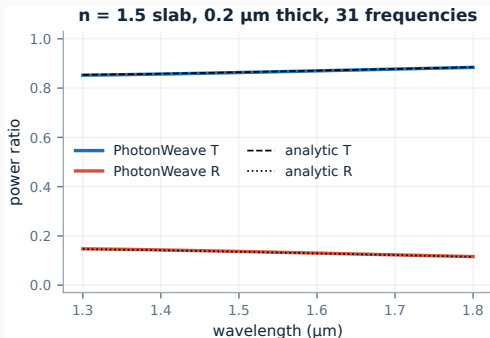
케이스별 반복 대신 한 launch.
결과는 개별 실행과 **bitwise 동일**

모니터 공유

평면 보간 + DFT 누적을
케이스 · 평면에 걸쳐 공유:
3.63–8.12×

정확도 검증

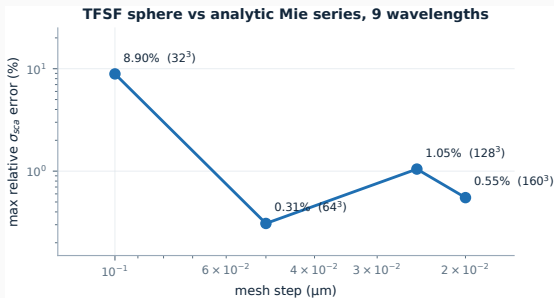
해석해와 비교 1: 유전체 박막의 R , T



동일한 공기 reference로 정규화, 반사는 복소 E/H incident subtraction. 최대 절대 오차 $|T - T_{\text{exact}}| = 1.53\text{e-}3$, 에너지 잔차 $|R + T - 1| = 4.44\text{e-}6$.

이 그림은 오늘 이 저장소에서 CPU float64로 다시 돌려 만든 결과입니다 (1.2초).

해석해와 비교 2: TFSF 구 산란 vs Mie 급수

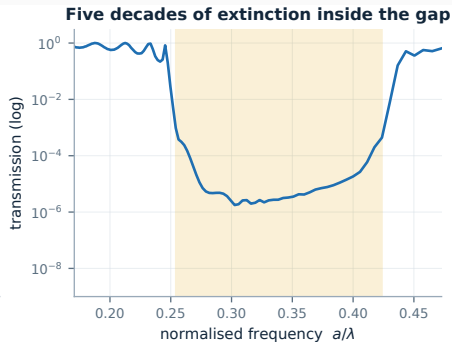
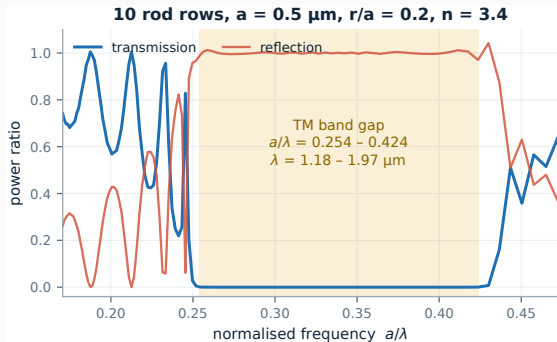


- 반지름 $0.3 \mu\text{m}$, $n = 1.5$ 구, $1.3\text{--}1.8 \mu\text{m}$ 9개 파장
- 도메인 크기 · 물리 PML 두께 · 120 fs 지속시간 고정
- $0.05 \mu\text{m}$ 메시에서 최대 오차 **0.31%**
- 오차가 단조 감소하지 않음: $0.025 \mu\text{m}$ 에서 1.05%로 증가
- staircase 경계 + TFSF 보정의 상호작용으로 해석 중

가장 좋은 행을 정확도 보장으로 쓰지 않습니다.

정량 결과에는 수렴 테스트가 필요합니다.

우리 연구 대상으로 직접 계산 1: photonic band gap



정사각 격자 유전체 rod ($a = 0.5 \mu\text{m}$, $r/a = 0.2$, $n = 3.4$) 10열, TM 편광. broadband one-way 평면 소스 + 주기 경계 + flux 모니터, 공기 reference로 정규화.

CPU float64로 두 번
(구조/reference) 실행해 약 25초. GPU에서는 이보다 훨씬 짧습니다.

우리 연구 대상으로 직접 계산 2: W1 선결함 도파로



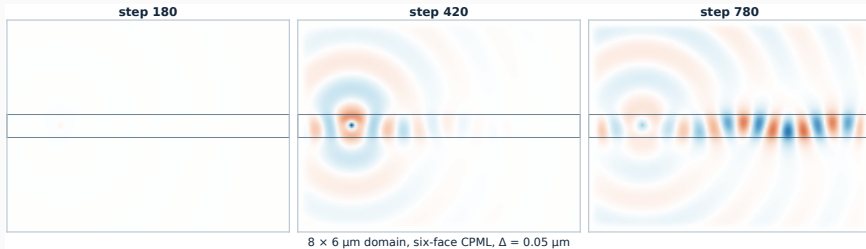
$a = 0.5 \mu\text{m}$, $r/a = 0.2$, $n = 3.4$, $\lambda = 1.55 \mu\text{m}$ (mid gap), $\Delta = 0.025 \mu\text{m}$, each panel scaled to its own peak

- 같은 결정에서 한 줄을 제거한 W1 채널
- 밴드갭 안쪽 $1.55 \mu\text{m}$ ($a/\lambda = 0.32$)로 여기
- 결정 안쪽으로는 evanescent, 채널을 따라 전파
- 160개 rod · 480×240 격자 · 6000 스텝을 CPU에서 약 20초

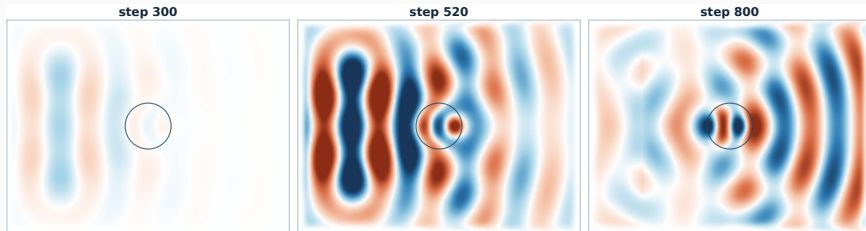
의미

랩에서 쓰는 PhC/metasurface 구조를 그대로 올려서, 스펙트럼과 필드를 같은 도구에서 얻습니다.

필드 시각화 예: 도파로와 산란체



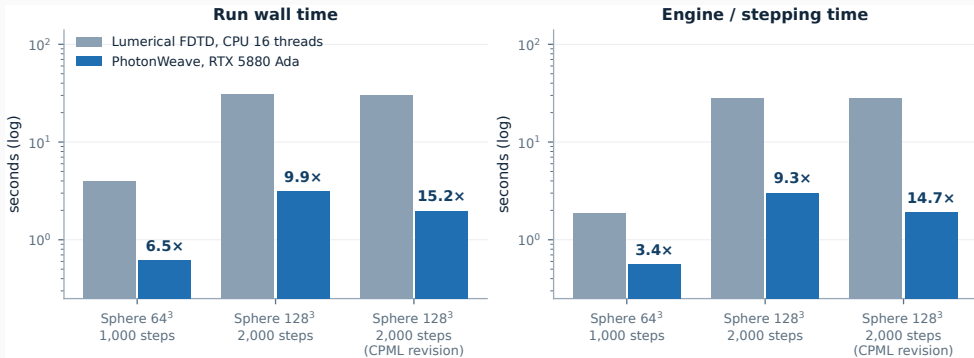
SiN 도파로, 다이폴 여기, 6면 CPML — 같은 계산이 UI의 Field visualizer에서 애니메이션으로 재생됩니다.



유전체 원기둥에 평면파 입사. 모든 그림은 이 저장소의 네이티브 솔버가 계산한 실제 E_z 스냅샷입니다.

성능

Lumerical FDTD와의 기록된 비교



6 μm 정육면체 도메인의 $n = 2$ 구 (반지름 0.6 μm), 1.55 μm Gaussian dipole, 포인트 모니터 1개. 각 값은 3회 측정의 중앙값입니다.

이 표를 읽는 법

과거 빌드의 기록이며 현재 릴리스의 검증된 speedup이 아닙니다. dipole 정규화·필드 보간·흡수 경계가 달라 동일 정확도 비교가 아니고, Lumerical GPU 엔진과는 측정하지 않았습니다. 20/37

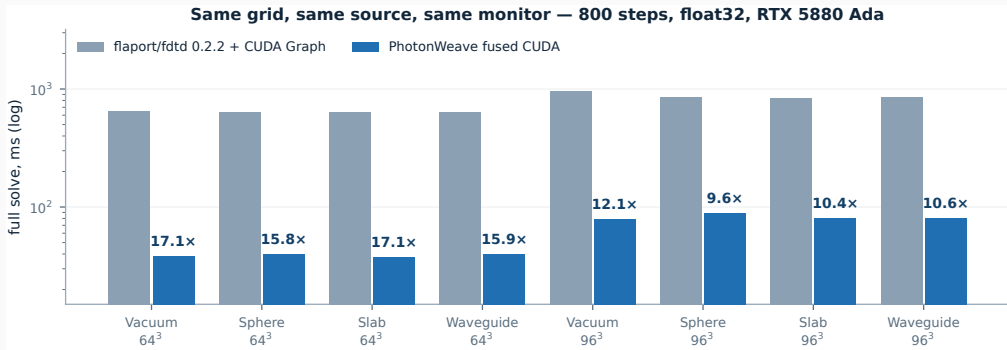
그래서 지금 다시 해야 하는 비교

필요한 비교	현재 상태
현재 PhotonWeave vs Lumerical CPU, 공통 정확도 목표	새 matched validation 필요
현재 PhotonWeave vs Lumerical GPU, 같은 RTX 5880	미측정
다구조 배치 / 역설계 throughput vs Lumerical	미측정
오픈소스 GPU 기준선 (flaport/fdtd + CUDA Graph) 대비	측정 완료
네이티브 CPU vs GPU, 동일 코드 경로 해석해 (박막, Mie) 대비 정확도	측정 완료 측정 완료, 한계 명시

랩미팅에서 주장하는 범위: "같은 물리 설정에서 우리 GPU 경로가 우리 CPU 경로와 공개 GPU 기준선보다 빠르다".

상용 솔버 대비 "동일 정확도 speedup"은 아직 주장하지 않습니다.

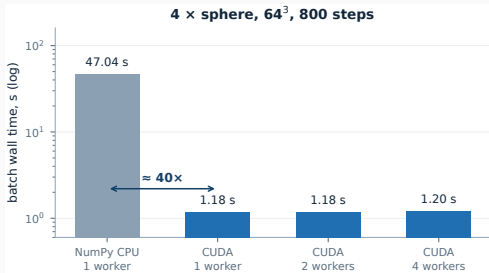
오픈소스 GPU 기준선 대비 단일 케이스



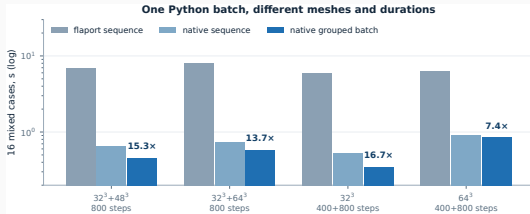
두 엔진에 동일한 voxel 유전율·시간 스텝·샘플 소스·포인트 모니터를 주고, 상용 기준선이 아닌 공개 코드에 CUDA Graph 어댑터까지 붙여 준 비교입니다. 8개 케이스 모두 포인트 트레이스 상대 L2 <0.04%.

최종 full field는 상위 PML 스텝일 차이로 동일하지 않습니다. 원자료에 오차를 모두 남겨 두었습니다.

배치: 같은 문제를 CPU와 GPU로

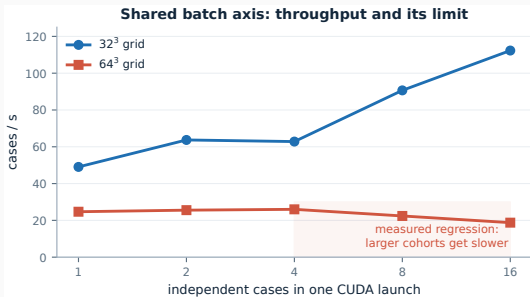


4개 독립 구 케이스, 64^3 , 800 스텝. 워커를 2/4 개로 늘려도 이 문제에서는 더 빨라지지 않습니다.



메시와 지속시간이 섞인 16 케이스를 한 Python 배치로. 호환 케이스를 자동 그룹화하고 입력 순서로 결과를 복원합니다.

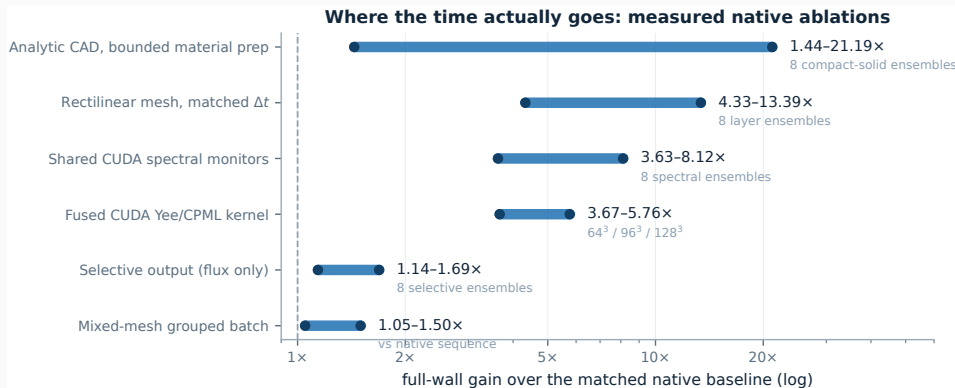
배치 축의 한계도 같이 봅니다



- 32³에서는 16 케이스까지 throughput이 증가 (**112 cases/s**)
- 64³에서는 8, 16 케이스에서 **오히려 느려짐**
- 해결책: cohort_size로 한 번에 묶는 케이스 수를 제한
- tune_tensor_batch()가 실제 측정으로 크기를 고르지만, **튜닝 자체가 비용**
- 64³에서는 튜닝 비용을 회수하는 데 같은 양상불 144~420회가 필요

출처: README.md · ensembles.json

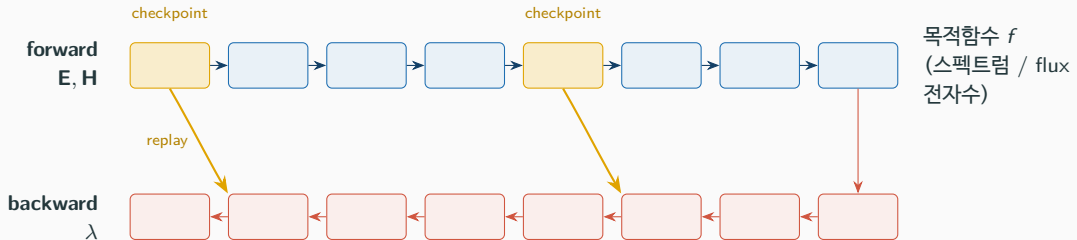
시간이 실제로 줄어든 곳



단일 커널 최적화만이 아니라, **호스트 준비 시간·불필요한 셀·저장하지 않아도 되는 출력**을 제거한 것이 실제 워크플로 시간에 더 크게 기여했습니다. 모든 행은 동일 조건의 네이티브 baseline과 비교한 full-wall 시간입니다.

미분 가능 FDTD와 역설계

왜 adjoint인가

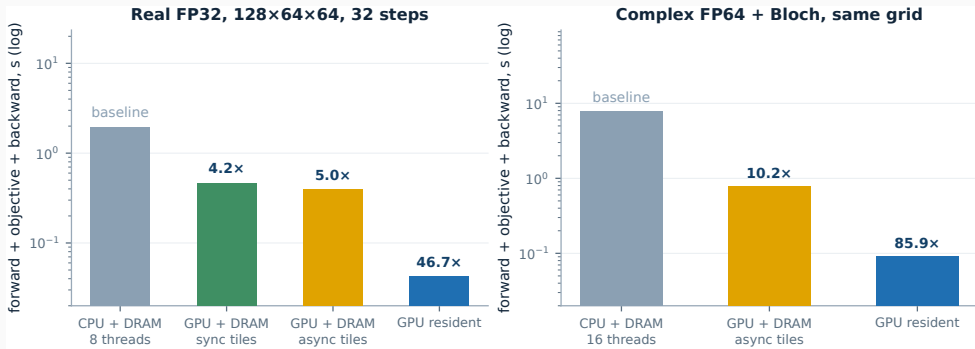


- 파라미터가 P 개여도 기울기 1회 = forward 1회 + backward 1회
- 유한차분 스윕은 $P + 1$ 회 계산 필요
- 전체 시간 이력 저장 대신 **checkpoint replay**로 메모리 제한

현재 미분 가능한 것

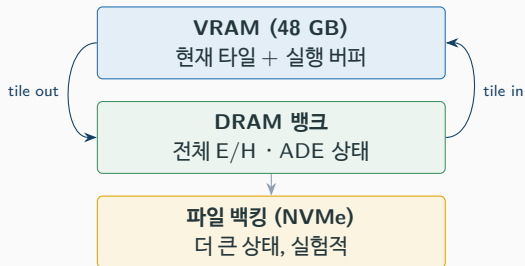
대각 ε , Torch geometry map, 고정 실수/Bloch 경계, Drude/Lorentz 파라미터 (ε_∞ , 진동자 세기, 공명, damping), 포인트 신호 · 스펙트럼 · 고정 검출 평면

adjoint 1회 비용: CPU vs GPU

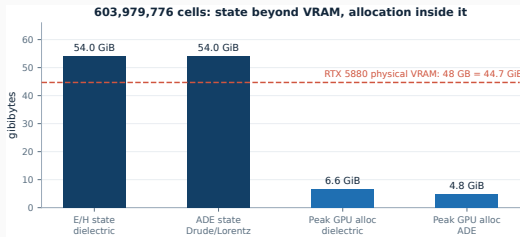


같은 코드 경로, 같은 격자($128 \times 64 \times 64$, 32 스텝), forward + 목적함수 + backward 전체 시간. 모든 모드가 신호·기울기 검사를 통과했고, CPU는 1/4/8/16 스레드 중 가장 빠른 설정을 사용했습니다. 이 문제는 VRAM 안에 들어갑니다. 48 GB를 넘는 영역의 속도나 외부 솔버 성능을 말하는 수치가 아닙니다.

VRAM보다 큰 문제를 푸는 방법

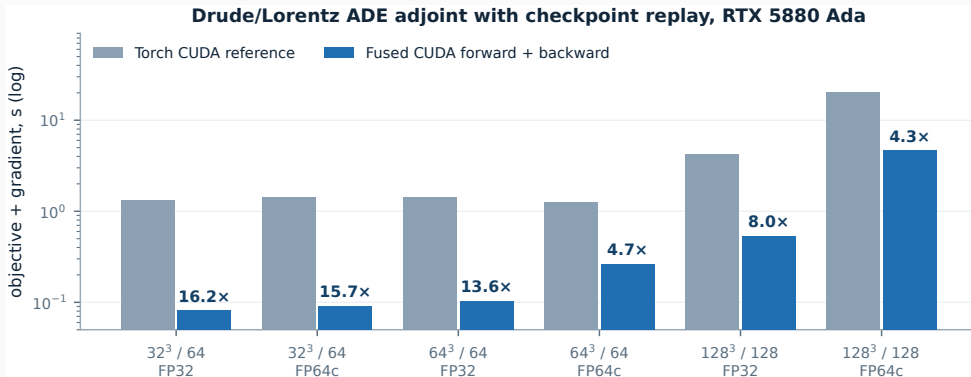


space-time tile: 한 칸 Yee 의존 cone을
halo로 포함해 손실 없이 분할



603,979,776 셀, E/H 54 GiB 상태에서 **10 스텝 전진 + 1차 기울기 검증 완료** (peak GPU 할당 6.6 GiB = 7.09 GB, 52분). 분산재료 ADE 버전은 4.8 GiB, 54.8분. 용량 증명이지 속도 우위가 아닙니다. 실제 응용 지속시간에서의 성능은 미검증.

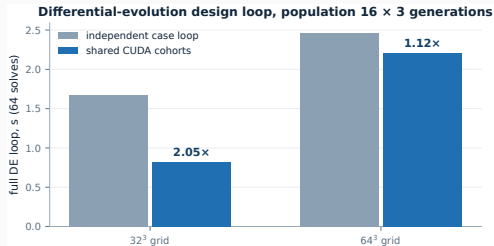
분산 재료 (Drude/Lorentz) 까지 미분



ADE의 P/Q 상태까지 checkpoint replay에 포함하고, 재료 기울기는 atomics 없이 블록 리덕션으로 모읍니다. 공유 파라미터는 compact 하게 유지되어 극점 수가 늘어도 메모리가 선형 증가하지 않습니다.

같은 연산 (전진+목적함수+역전파) 을 우리 Torch 경로와 비교한 내부 측정입니다.

설계 루프: 지금 돌아가는 형태



블랙박스 (미분 없는) differential evolution 루프,
population 16 × 3세대 = 64회 forward.

```
import torch
from photonweave import (DifferentiableSimulation,
                          AdjointOptions, smooth_sphere_epsilon)

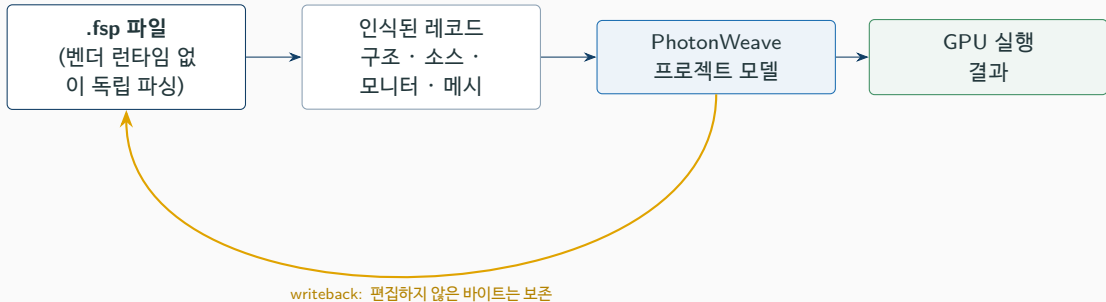
model = DifferentiableSimulation(project,
                                 AdjointOptions(checkpoints=4, storage="host"))
radius = torch.nn.Parameter(
    torch.tensor(0.25, dtype=torch.float64))
opt = torch.optim.Adam([radius], lr=0.003)

for step in range(iterations):
    opt.zero_grad()
    epsilon = smooth_sphere_epsilon(
        project.region, radius, width=0.09, inside=3.0)
    result = model(epsilon) # forward
    loss = result.signals[:, 0].abs().square().mean()
    loss.backward() # discrete adjoint
    opt.step()
```

examples/differentiable_design.py의 실제
코드. 스트리밍 실행은 StreamedSimulation으로
같은 자리에 들어갑니다. 포트 정규화 목적함수 검증은
진행 중입니다.

상호운용, 현황, 다음 단계

기존 .fsp 파일을 버리지 않기



- box, 회전 타원체/원기둥, 부분 링, 단순 polygon extrusion import
- 메시 간격·span·CAD/PML 경계·CFL, 소스 대역/위상, 모니터 스펙트럼 writeback

지원하지 않는 물리는 변환을 막습니다. 외부
도구의 새 레코드 수용·그룹·결과 포함 파일·
일반 호환성은 미해결.

오픈소스 GPU FDTD 지형에서의 위치

	백엔드	동일 GPU 배치	adjoint	비고
PhotonWeave	PyTorch + native CUDA	공유 CUDA launch, cohort 분할, 측정 기반 선택	부분: 실수 유전체 Yee/CPML + 분산 파라미터	브라우저 CAD, FSP 서브셋, 멀티폴 ADE
FDTDx	JAX	JAX 조합	있음 (reversible/checkpoint)	역설계는 우리보다 앞섬
fdtdz	JAX + 전용 CUDA	미검증	등록된 VJP 규칙 없음	매우 빠른 전용 범위, z 크기 제약
flaport/fdtd	NumPy / PyTorch	공개 Grid는 1 케이스	기본 백엔드는 grad 비활성	우리가 기반으로 사용·출처 표기
fdtd3d	C++ / CUDA / MPI	단일 문제 도메인 분할	문서화 없음	단일 격자 MPI는 우리에게 없음

정직하게: FDTDx, fdtdz, fdtd3d와는 같은 GPU에서 속도를 측정하지 못했습니다 (현재 워크스테이션에 Linux CUDA 환경 부재). 기능 차이는 속도 우위가 아닙니다.

아직 안 되는 것 (질문 받기 전에 먼저)

물리

- 이득 매질, 비선형, 이방성 미지원
- 대칭/반대칭 경계, PEC/PMC 미구현
- 경사 입사 one-way/TFSF, 도파로 모드 소스 미구현
- far-field 변환, mode port, S-parameter 미구현
- conformal/subpixel은 실험적 (기본은 staircase)

워크플로 · 검증

- 체크리스트 1,661행 중 구현 160 / 부분 231 / 미구현·미검증 1,270
(속성 수이며 제품 완성도 비율이 아님)
- 단일 격자 MPI 분할 없음
- 응용 수준 역설계 (mode/port 정규화 목적함수) 검증 미완료
- 상용 솔버와 동일 정확도 조건의 재측정 필요
- 공개 배포는 라이선스 검토 미완료

문서: docs/FEATURE_CHECKLIST.md · docs/PARITY_ROADMAP.md ·
docs/RELEASE_REVIEW.md

다음 단계

단기

- 동일 정확도 기준의 상용 솔버 재측정
- mode/port 정규화 목적함수
- subpixel 정확도 정리

중기

- 전 물리 미분 가능화(분산·경계 포함)
- 통합 메모리 정책 (resident/streamed 자동 선택)
- 실제 응용 지속시간의 대형 역설계

랩 적용

- PhC/metasurface 스위치를 배치로 이전
- 측정 n/k 데이터 피팅 후 프로젝트에 보관
- GPU 워크스테이션에 SSH 터널로 공유 사용

필요한 구조·조건을 알려주시면 그 케이스로 먼저 벤치마크하고, 정확도 게이트를 붙여서 가져오겠습니다.

요약

상용 FDTD와 같은 작업 순서를 유지한 브라우저 + Python
워크벤치

같은 물리 설정에서 우리 CPU 경로와 공개 GPU 기준선보다 빠른
CUDA 엔진

설계 루프를 위한 배치 실행과 discrete adjoint, VRAM을 넘는
메모리 계층

속도 수치는 조건과 한계를 함께 기록 — 상용 솔버 동일정확도
비교는 진행 중

백업: 수치 규약

- 길이·파장은 μm , API 시간 배열은 초
- 진폭은 reduced field (보정된 V/m 아님)
- E와 H는 시공간에서 엇갈려 있으므로 곧바로 곱해 Poynting으로 쓰지 않음
- 포인트 모니터: FFT bin 또는 사용자 지정 범위 DFT, None/Start/End/Full/Hann apodization
- 평면 모니터: 6성분 복소장 + 부호 있는 flux, reference 정규화 필요
- 재료: 상수 $n \geq 1$, 수동 등방 Drude/Lorentz 최대 16 극점
- 내장 Si/SiN/SiO₂ 값은 편집 가능한 근사치이며 광학상수 DB가 아님
- 측정 n/k 샘플을 수동 극점으로 피팅하고 오차를 명시
- 모든 벤치마크는 3회 반복 중앙값, warmup 후 측정, cold compile 제외
- 3회 반복은 신뢰구간을 주지 않음

백업: 이 슬라이드의 그림을 다시 만드는 법

```
# figures: charts + real CPU simulations
python docs/presentation/make_figures.py
```

```
# slides (XeLaTeX, Korean)
cd docs/presentation
xelatex photonweave-lab-meeting.tex
```

```
# or
bash docs/presentation/build.sh
```

- 필드 그림 4종은 실행 때마다 **실제로 다시 계산**됩니다 (CPU, 총 1분 이내)
- 성능 차트의 모든 수치는 README와 docs/validation의 기록값
- UI 화면은 로컬 서버(`python -m photonweave.cli serve`)의 실제 캡처

GPU 측정	NVIDIA RTX 5880 Ada Generation 48 GB, Windows, PyTorch 2.10
보조 GPU	RTX 3060 (복소 FP64 CR 관련 측정)
CPU 비교	Xeon w3-2435 (8/16 스레드), i7-12700 (8 스레드)
외부 기준선	flaport/fdtd 0.2.2 + 자체 CUDA Graph 어댑터 + 동일 fused 관측자
상용 기록	Lumerical FDTD CPU 1 프로세스 16 스레드, API 버전 문자열 8.31.3633 (v241)
정확도 게이트	포인트 트레이스 1%, flux 2%, 복소 평면 DFT 1% (외부), 네이티브 간 bitwise

Lumerical 계산 결과·필드·프로젝트 파일은 저장소에 포함하지 않으며, 타이밍 요약만 기록했습니다. 해당 표는 공개 배포 전 라이선스 검토 대상입니다.